МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп’ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ №8

Тема: «Алгебраїчні типи даних “по-Python’івськи”»

Варіант 15 — Абстрактне дерево запитів

Виконала:
студентка 2 курсу групи ФЕП-22
Сулятицька Христина

Перевірив:
Остріков Данило Андрійович

Львів – 2026

Мета роботи

Мета роботи — ознайомитися з алгебраїчними типами даних у Python, навчитися моделювати рекурсивні структури за допомогою незмінних dataclass-вузлів, Union та TypeAlias, застосовувати Literal і Protocol, а також реалізовувати обробку структур через match/case та перевіряти типи засобом mypy.

Теоретичні відомості

Алгебраїчний тип даних (ADT) дає змогу описати значення як суму кількох можливих варіантів. У Python таку модель можна реалізувати набором класів dataclass та об'єднанням Union. Декоратор frozen=True забороняє змінювати поля створених вузлів.

TypeAlias задає зрозуміле ім'я для складного типу. Literal обмежує допустимі значення типу конкретними літералами. Protocol описує структурний контракт: об'єкт відповідає йому, якщо має потрібні методи або атрибути, незалежно від успадкування.

Конструкція match/case виконує структурний розбір об'єкта. Для цього завдання вона дає змогу рекурсивно обробляти вузли Field, Eq, And, Or і Not без довгих ланцюгів if/elif.

Хід роботи

1. Аналіз індивідуального варіанта

За варіантом 15 необхідно побудувати абстрактне дерево запитів із вузлами Field, Eq, And, Or, Not. Основне завдання — побудувати та обчислити простий фільтр для словників даних. Додатково необхідно реалізувати TypeAlias для Query, Literal для операторів, а також функції render_query() та eval_query().

2. Створення моделі ADT

У файлі model.py описала незмінні класи Field, Eq, And, Or і Not за допомогою @dataclass(frozen=True). Тип Query оголосила через Union та TypeAlias. Для оператора порівняння використала Literal["=="], а для логічних операторів — Literal["and", "or", "not"].

Також оголосила Protocol SupportsRecordLookup із методом __getitem__. Він задає структурний контракт для об’єктів, з яких можна отримувати значення за текстовим ключем. Завдяки цьому eval_query працює зі словниками без прив’язки до конкретного класу.

3. Реалізація обчислювача

У evaluator.py реалізувала чисту функцію eval_query(query, record). За допомогою match/case вона розпізнає кожен вузол дерева. Eq порівнює значення поля із заданим значенням, And вимагає істинності обох підзапитів, Or — хоча б одного, а Not інвертує результат вкладеного запиту.

Якщо запис не містить потрібного поля, функція генерує QueryEvaluationError із назвою відсутнього поля. Логічні And та Or використовують коротке замикання. Вхідні вузли та словник не змінюються.

4. Формування текстового представлення

У pretty.py реалізувала render_query(). Функція рекурсивно формує читабельний текст запиту, використовуючи AND, OR та NOT. Дужки зберігають структуру вкладених логічних операцій.

5. Демонстрація роботи

У main.py створила список словників із даними людей та сформувала вкладений запит: місто має дорівнювати Lviv, поле active — True, а вік не повинен дорівнювати 20 або 22. Після цього програма виводить текст запиту та записи, які йому відповідають.

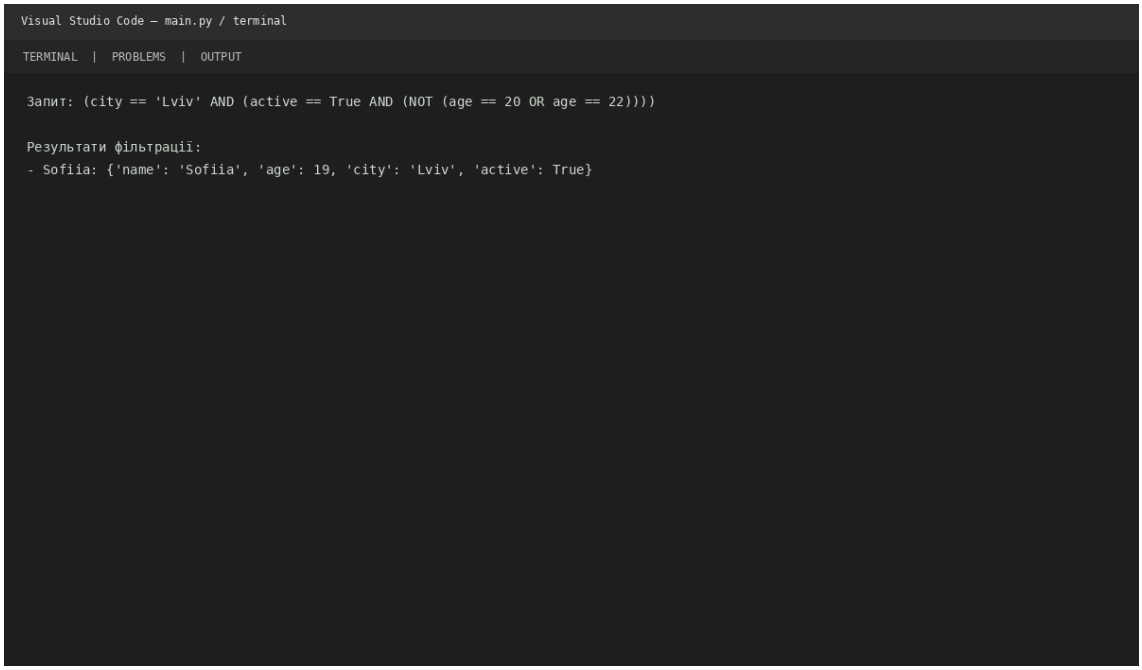


Рисунок 1 — Демонстрація запуску main.py

6. Тестування

Для перевірки написала pytest-тести на окремі вузли Eq, And, Or, Not, вкладені запити, текстове представлення, незмінність вузлів, відсутнє поле та коротке замикання логічних операцій.

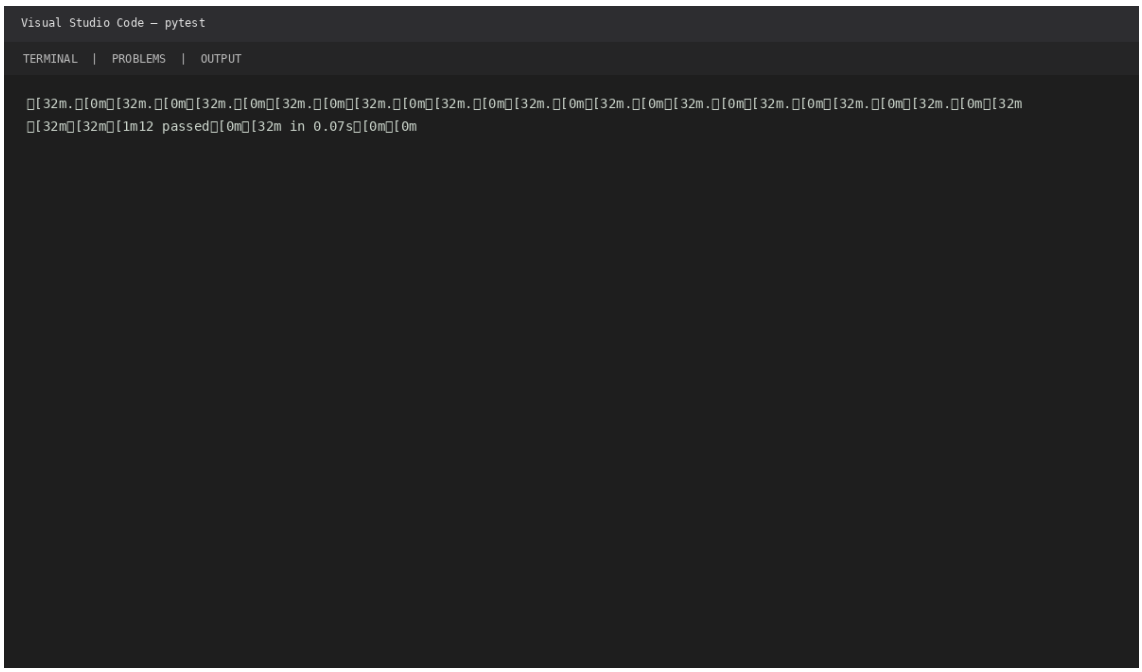


Рисунок 2 — Результат виконання pytest

Під час перевірки всі 12 тестів завершилися успішно.

7. Перевірка типів

У проєкті додала mypy.ini зі строгими налаштуваннями, зокрема strict=True, warn_return_any=True, warn_redundant_casts=True, warn_unused_ignores=True та disallow_any_generics=True. Команда перевірки відповідно до методичних вказівок: mypy --strict .

У середовищі підготовки звіту mypy не встановлений, тому фактичний запуск статичної перевірки не підтверджений. Для перевірки на комп'ютері необхідно встановити залежності командою python -m pip install -r requirements.txt, а потім виконати mypy --strict .

Висновок

Я розробила абстрактне дерево запитів для фільтрації словників даних відповідно до 15 варіанта. Створила незмінні dataclass-вузли Field, Eq, And, Or і Not, описала рекурсивний тип Query через Union та TypeAlias, використала Literal і Protocol. Реалізувала чисті функції eval_query() та render_query() із застосуванням match/case. Передбачила обробку відсутніх полів, перевірила вкладені запити, незмінність даних і коротке замикання логічних операцій. Усі 12 автоматизованих тестів пройшли успішно. Також підготувала конфігурацію для суворої статичної перевірки типів mypy.